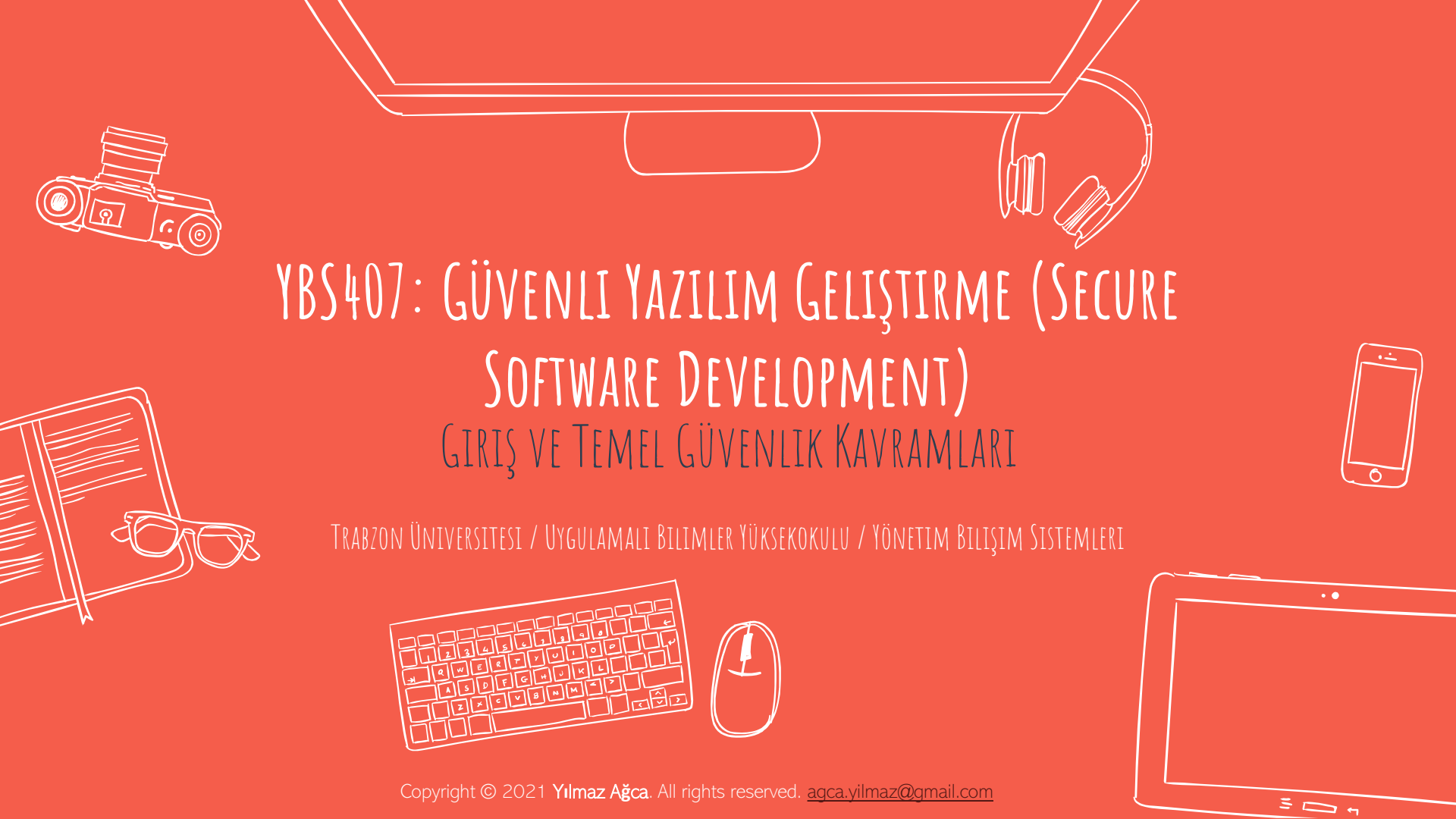


YBS407: Güvenli Yazilim Geliştirme (Secure Software Development)

Giriş ve Temel Güvenlik Kavramları

Trabzon Üniversitesi / Uygulamali Bilimler Yüksekokulu / Yönetim Bilişim Sistemleri

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Konular

1. Bilgi Güvenliği ve Etik nedir?
2. CIA Üçgeni nedir?
   a) Gizlilik (Confidentiality)
   b) Bütünlük (Integrity)
3. Etik ve Hukuki Temeller

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Yazılım Geliştirme

Güvenli yazılım geliştirme dünyasının temel taşını oluşturan

1. **güvenlik açığı**,
2. **tehdit**,
3. **risk** ve
4. **saldırı yüzeyi**

kavramları, tüm güvenlik stratejilerinin üzerine kurulduğu temel sözlüğü oluşturur.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenlik Açığı (Vulnerability)

Güvenlik açığı, bir yazılımın kodunda, tasarımında, mimarisinde veya yapılandırmasında bulunan ve bir saldırgan tarafından istismar edilerek sistemin güvenliğini ihlal etmeye olanak tanıyan **zayıflık veya kusurdur**.

Örnek: Bir web uygulamasında kullanıcıdan alınan girdinin (input) yeterince doğrulanmaması (sanitization yapılmaması) sonucu ortaya çıkan **SQL Injection** veya **Cross-Site Scripting (XSS)** açıkları.

Temel Nedenleri: İnsan hataları, zaman baskısı, karmaşık kod tabanları veya yetersiz test süreçleri.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

SQL Injection (SQLi) Örneği

```python
from flask import Flask, request
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # TEHLİKELİ: Girdi doğrudan SQL sorgusuna ekleniyor!
    query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"

    cursor.execute(query)
    user = cursor.fetchone()

    if user:
        return "Giriş Başarılı!"
    return "Hatalı Giriş!"
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

SQL Injection (SQLi) Örneği

Saldırı Senaryosu (İstismar): Bir saldırgan kullanıcı adı kısmına normal bir isim yazmak yerine şunu yazarsa:

```
admin' --
```

Oluşan SQL sorgusu şu hale gelir:

```
SELECT * FROM users WHERE username = 'admin' --' AND password = '... '
```

Buradaki `--` işareti SQL'de yorum satırı anlamına geldiği için sorgunun geri kalanını (şifre kontrolünü) tamamen devre dışı bırakır. Saldırgan şifre bilmeden sisteme `admin` olarak giriş yapabilir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

SQL Injection (SQLi) Örneği

```
@app.route('/login', methods=['POST'])
def login_secure():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # GÜVENLİ: Parametrik yapı (Placeholder ? kullanımı)
    query = "SELECT * FROM users WHERE username = ? AND password = ?"

    cursor.execute(query, (username, password))
    user = cursor.fetchone()

    if user:
        return "Giriş Başarılı!"
    return "Hatalı Giriş!"
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Cross-Site Scripting (XSS) Örneği

XSS, bir web uygulamasının kullanıcıdan aldığı girdiyi yeterince "kaçış (escaping)" işleminden geçirmeden doğrudan tarayıcıya (HTML çıktı olarak) yansıtması durumunda ortaya çıkar.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Cross-Site Scripting (XSS) Örneği

```
1.  from flask import Flask, request, render_template_string
2.
3.  @app.route('/comment', methods=['POST'])
4.  def comment():
5.      user_comment = request.form['comment']
6.
7.      # TEHLİKELİ: Kullanıcı girdisi filtrelenmeden doğrudan HTML içine yazılıyor
8.      html_template = f"<div><h3>Kullanıcı Yorumu:</h3><p>{user_comment}</p></div>"
9.
10.     return render_template_string(html_template)
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Cross-Site Scripting (XSS) Örneği

Saldırı Senaryosu (İstismar): Bir kullanıcı yorum kısmına normal bir metin yerine şu JavaScript kodunu gönderirse:

```
<script>alert(document.cookie);</script>
```

Sayfa yüklendiğinde bu script tarayıcı tarafından çalıştırılır ve diğer kullanıcıların çerezleri (oturum bilgileri) çalınabilir veya kötü amaçlı başka sitelere yönlendirilebilirler.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Cross-Site Scripting (XSS) Örneği

```
@app.route('/comment', methods=['POST'])
def comment_secure():
    user_comment = request.form['comment']

    # GÜVENLİ: Modern şablon motorları (Jinja2) değişkenleri otomatik olarak escape eder.
    # Eğer özel bir HTML filtremesi gerekiyorsa bleach gibi kütüphanelerle sanitize edilmelidir.
    return render_template_string("<div><h3>Kullanıcı Yorumu:</h3><p>{{ user_comment }}</p></div>",
user_comment=user_comment)
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Tehdit (Threat)

Tehdit, bir sistemin güvenliğine zarar verme, veri sızdırma veya sistemi hizmet dışı bırakma (**CIA Üçgeni** olan Gizlilik, Bütünlük ve Erişilebilirlik ilkelerini bozma) potansiyeli taşıyan **her türlü tehlike veya aktördür**.

Tehditler insan kaynaklı (örn. hackerlar, içerideki kötü niyetli çalışanlar) olabileceği gibi doğal afetler veya donanımsal arızalar gibi sistemik de olabilir.

Örnek: Uygulamanızdaki bir giriş formunu hedef alarak şifre kırma (brute-force) denemeleri yapmak veya veritabanını ele geçirmeye çalışmak birer tehdittir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Risk (Risc)

Risk, bir **tehdidin**, mevcut bir **güvenlik açığını** istismar etme olasılığı ile bunun sonucunda ortaya çıkacak **zararın (etkinin)** birleşimidir. Temel formülü şu şekilde özetleyebiliriz:

$$\text{Risk} = \text{Olasılık} \times \text{Etki}$$

Açıklama: Eğer bir sistemde kritik bir açık varsa ama bu açığa dışarıdan erişilemiyorsa risk düşüktür.

Ancak hem çok kritik bir açık varsa hem de internete açık bir alandaysa risk çok yüksektir.

Güvenlik mühendisliğinin temel amacı riskleri tamamen sıfırlamak değil, kabul edilebilir seviyeye indirmektir (Risk Yönetimi).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Saldırı Yüzeyi (Attack Surface)

Saldırı yüzeyi, bir yazılımın veya sistemin dışarıdan (yetkisiz kullanıcılar veya zararlı yazılımlar tarafından) **saldırıya uğrayabileceği tüm giriş noktalarının, bileşenlerin ve yolların toplamıdır.**

Neleri Kapsar?

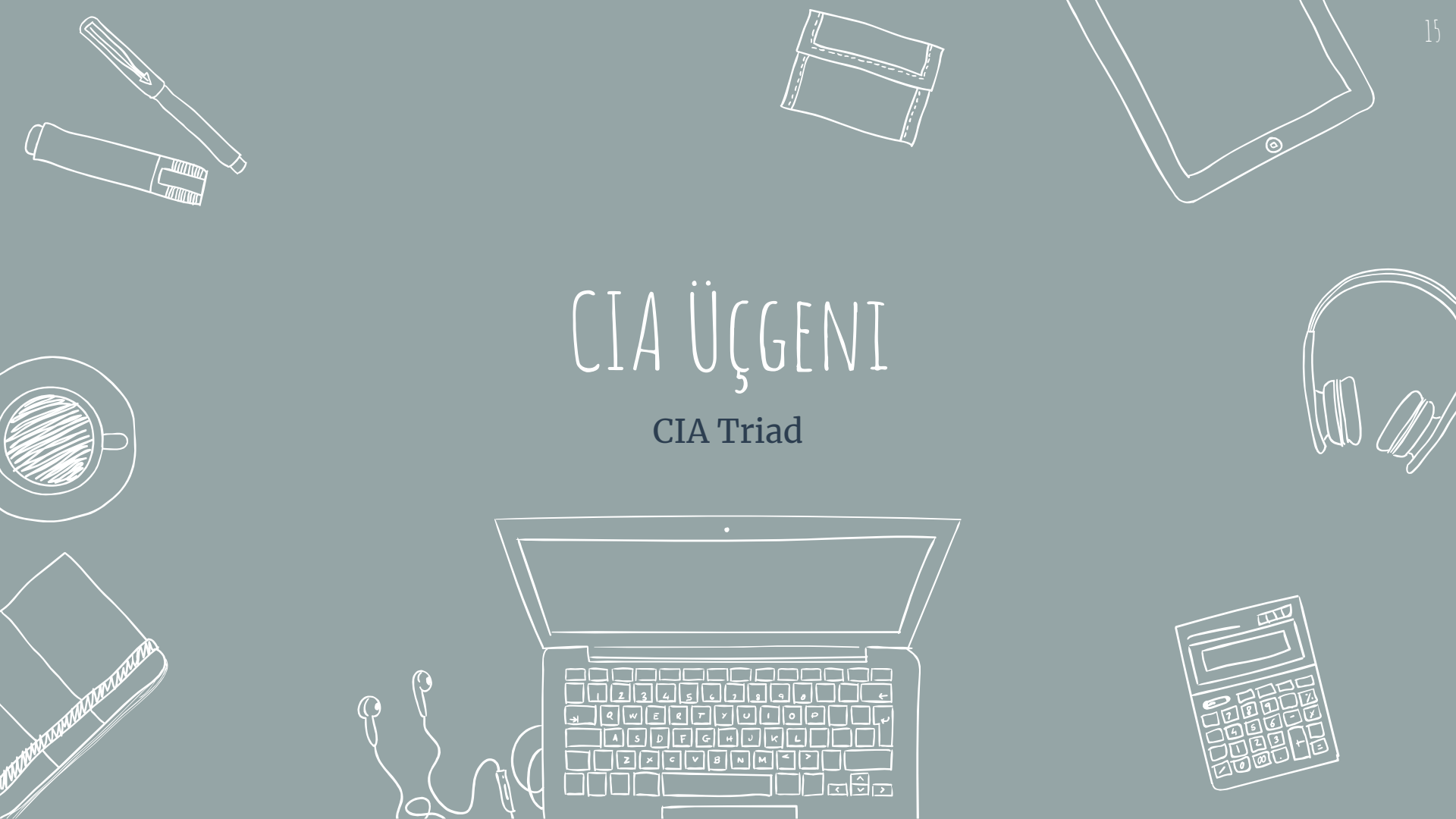
- Açık ağ portları ve servisler
- Kullanıcı girdisi alan tüm web formları ve API uç noktaları (Endpoints)
- Üçüncü taraf kütüphaneler ve entegrasyonlar
- Yetkilendirilmiş kullanıcı hesapları ve arayüzler

Altın Kural: Saldırı yüzeyi ne kadar genişse, sistemin güvenliğini sağlamak o kadar zorlaşır. Bu nedenle **"Saldırı yüzeyini daraltmak" (Attack Surface Reduction)** güvenli yazılım geliştirmenin en temel hedeflerinden biridir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

CIA Üçgeni

CIA Triad



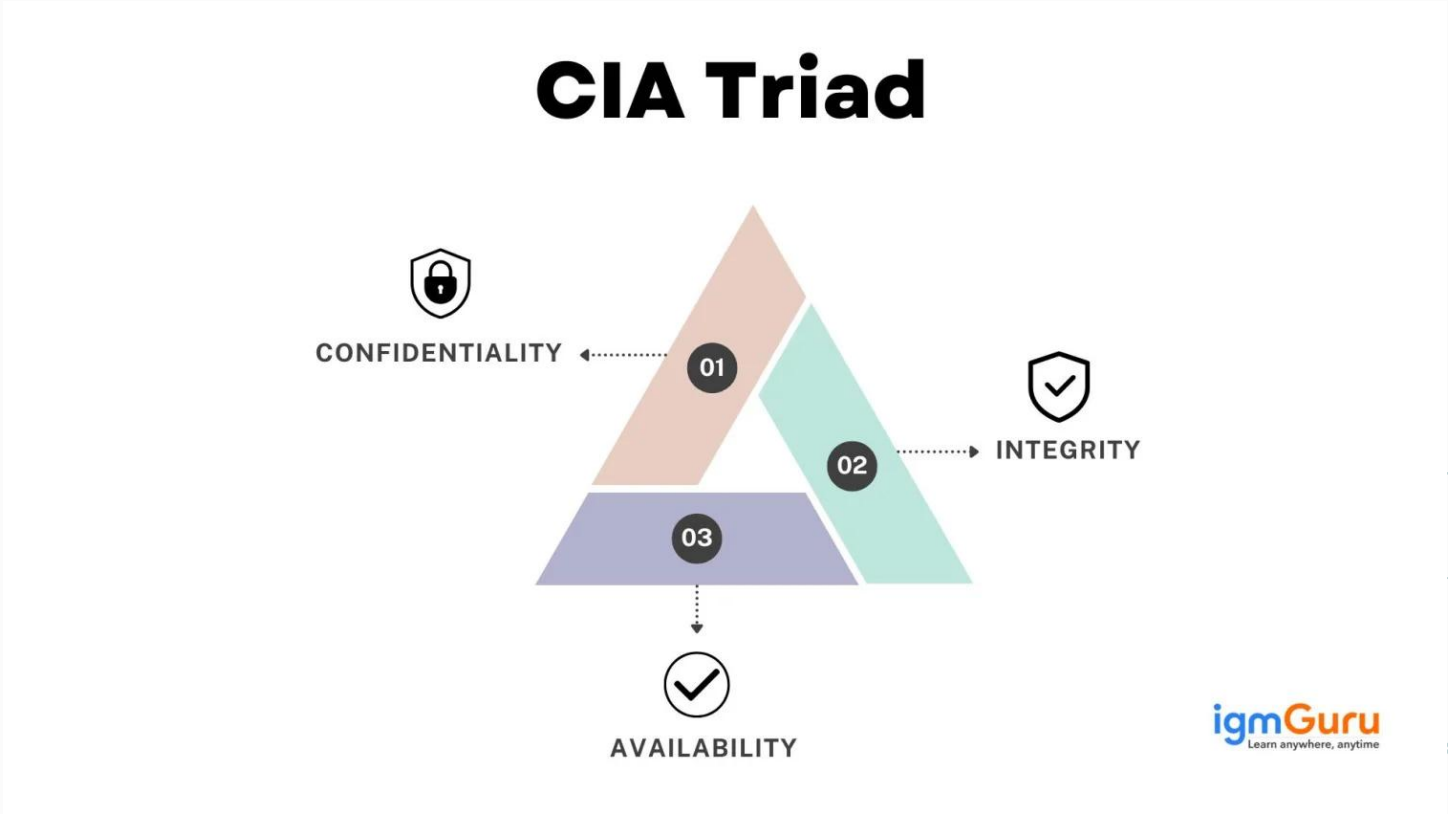
CIA Üçgeni

CIA Üçgeni, bilgi güvenliğinin temelini oluşturan ve bir kurumun veya sistemin güvenlik politikasını şekillendiren üç temel bileşenin baş harflerinden oluşan bir modeldir.

Bilgi güvenliği profesyonelleri, sistemleri korurken ve zafiyetleri analiz ederken her zaman bu üç temel ilkeyi göz önünde bulundurur.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

CIA Üçgeni nedir?



Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Gizlilik (Confidentiality)

Gizlilik (Confidentiality), hassas veya özel bilgilerin yalnızca bunlara erişmeye **yetkisi olan** kişiler, sistemler veya süreçler tarafından okunabilmesini ve görülebilmesini sağlayan temel bir güvenlik ilkesidir.

Diyelim ki mobil bankacılık uygulamanıza giriş yapacaksınız. TC kimlik numaranızı ve şifrenizi girdiniz. Bankanın sistemi arka planda şunu kontrol eder: *"Bu verileri görmeye ve bu hesabı yönetmeye gerçekten yetkisi olan kişi bu mu?"* Eğer doğru şifreyi girerseniz sistem size hesap özetinizi gösterir (Gizlilik sağlanmıştır). Ancak şifrenizi bilmeyen bir başkası telefonunuzu eline alsa bile uygulamaya giremez ve hesap detaylarınızı göremez. Burada şifre, gizliliği koruyan bir bariyerdir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

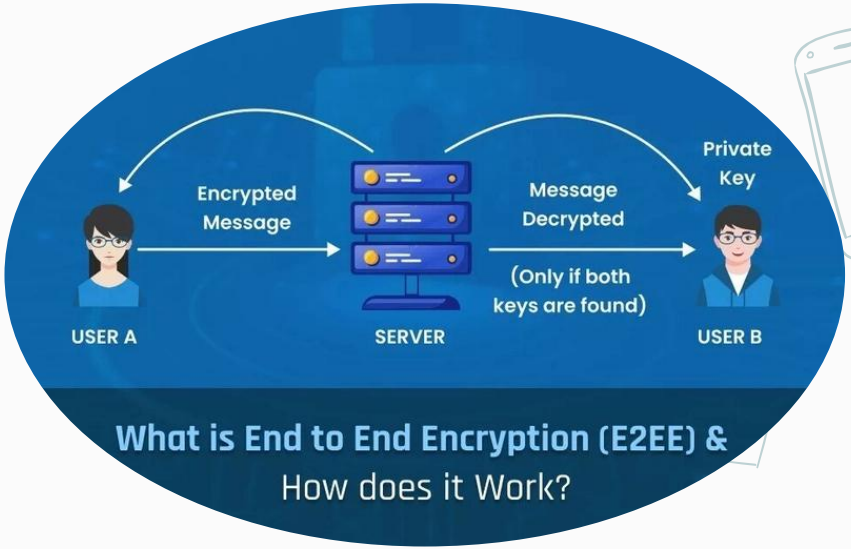
Gizlilik (Confidentiality)

Bilişim Dünyasından Örnek: Şifrelenmiş E-posta (End-to-End Encryption)

İki kişi arasında gönderilen gizli bir mesajı düşünelim. Mesaj internet üzerinden seyahat ederken aradaki servis sağlayıcılar, hackerlar veya internet ağını dinleyen kötü niyetli kişiler bu veriyi ele geçirebilir.

Eğer mesaj **şifrelenmemiş (plain text)** ise okudukları anda gizlilik ihlal edilmiş olur.

Ancak **uçtan uca şifreleme(End-to-End Encryption)** kullanılıyorsa, mesaj sadece gönderenin ve alıcının cihazındaki anahtarla çözülebilir. Aradaki herkes mesajı görse bile sadece anlamsız karakterler



Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Gizlilik (Confidentiality)

Bilişim Dünyasından Örnek: Şifrelenmiş E-posta (End-to-End Encryption)

İki kişi arasında gönderilen gizli bir mesajı düşünelim. Mesaj internet üzerinden seyahat ederken (örneğin bir mesajlaşma uygulamasında) aradaki servis sağlayıcılar, hackerlar veya internet ağını dinleyen kötü niyetli kişiler bu veriyi ele geçirebilir.

Eğer mesaj **şifrelenmemiş (plain text)** ise okudukları anda gizlilik ihlal edilmiş olur.

Ancak **uçtan uca şifreleme(End-to-End Encryption)** kullanılıyorsa, mesaj sadece gönderenin ve alıcının cihazındaki anahtarla çözülebilir. Aradaki herkes mesajı görse bile sadece anlamsız karakterler

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)

Bütünlük (Integrity), verinin veya bilginin yetkisiz kişiler tarafından değiştirilmesini, bozulmasını ya da tahrif edilmesini önleyerek **doğruluğunu ve güvenilirliğini** korumayı amaçlayan temel bir güvenlik ilkesidir.

Bütünlük sağlandığında, bilginin kaynağında çıktığı andan hedef ulaştığı ana kadar hiçbir müdahaleye uğramadığı garanti edilir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)

Diyelim ki bir arkadaşınıza internet bankacılığı üzerinden **100 TL** gönderdiniz.

Eğer sistemde veri bütünlüğü korunmuyorsa, kötü niyetli bir kişi bu veri transferine araya girip müdahale edebilir ve tutarı **10.000 TL** olarak değiştirebilir.

Karşı tarafa giden tutarın değişmesi veya işlem kaydının bozulması, bütünlük ilkesinin ihlal edildiğini gösterir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)

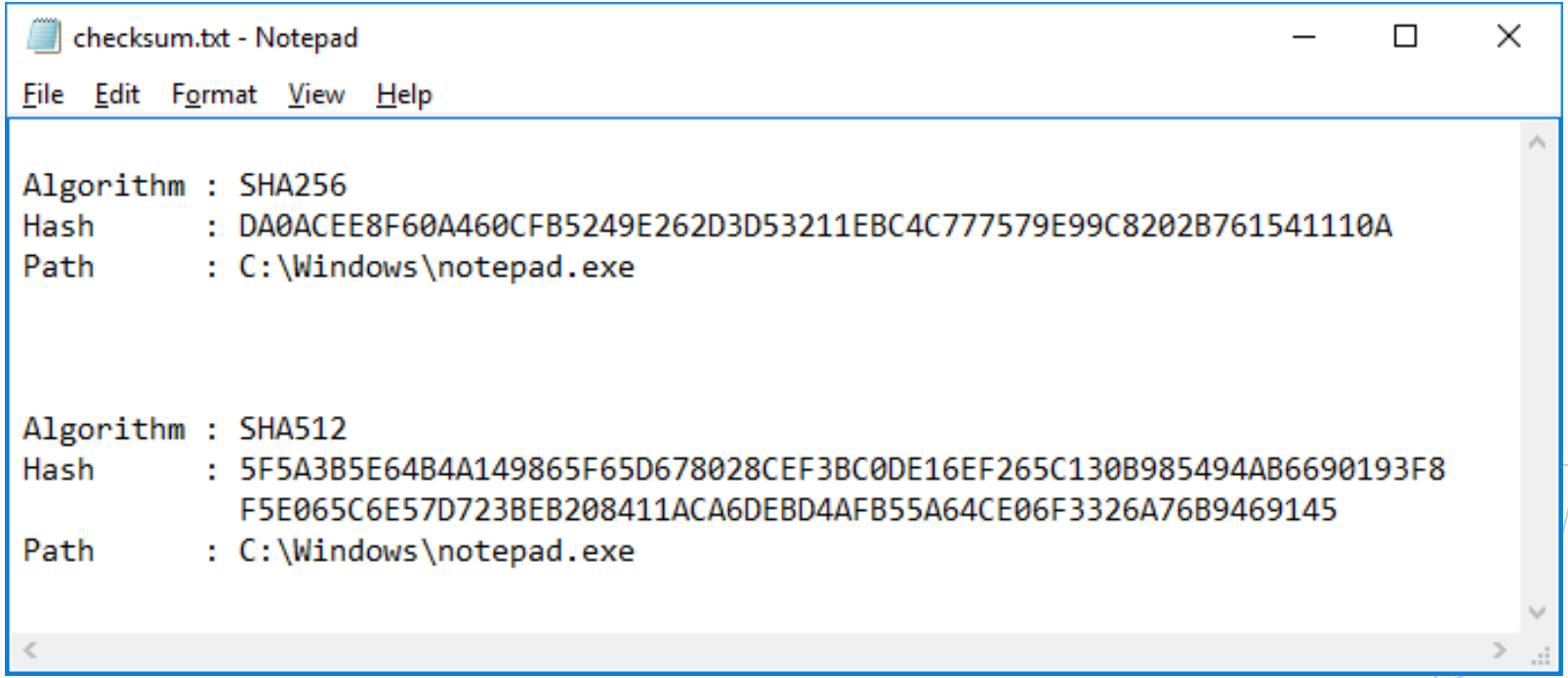
İnternetten resmi bir kaynaktan açık kaynaklı bir yazılım indirdiğinizi düşünün. Yazılımı sunan site, dosyanın **SHA-256** gibi matematiksel bir **hash değerini** (parmak izini) de paylaşır.

Siz dosyayı indirip kendi bilgisayarınızda bir hash hesaplaması yaptığınızda çıkan sonuç, resmi sitedekiyle birebir uyuşuyorsa dosyanın bütünlüğü korunmuştur.

Eğer araya bir hacker girip yazılımın içine virüs enjekte ettiyse, dosyanın içeriği değişeceği için hash değeri de tamamen farklı çıkacaktır; bu da bütünlüğün bozulduğunu ele verir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)



Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)

Windows 11 **Başlat** menüsüne sağ tıklayın ve **Terminal** veya **PowerShell**'i açın.

Şu komutu yazın ve Dosya_Yolu kısmını kontrol etmek istediğiniz dosyanın gerçek yoluyla değiştirin:

```
Get-FileHash -Path "C:\dosya\yolu\dosyaadi.iso" -Algorithm SHA256
```

(Algoritma olarak SHA256 yerine MD5 veya SHA1 de yazabilirsiniz).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)

macOS'ta Hash Kontrolü

macOS üzerinde en yaygın kullanılan algoritma **SHA-256**'dır. **Terminal** uygulamasını açarak şu komutları kullanabilirsiniz:

SHA-256 için:

```
shasum -a 256 /dosya/yolu/dosyaadi.iso
```

MD5 için:

```
md5 /dosya/yolu/dosyaadi.iso
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bütünlük (Integrity)

Linux'ta Hash Kontrolü

Linux dağıtımlarında (Ubuntu, Fedora, Mint vb.) her algoritmanın kendi özel komut aracı vardır. Terminali açıp şu komutları çalıştırabilirsiniz:

SHA-256 için:

```
sha256sum /dosya/yolu/dosyaadi.iso
```

MD5 için:

```
md5sum /dosya/yolu/dosyaadi.iso
```

SHA-1 için:

```
sha1sum /dosya/yolu/dosyaadi.iso
```

💡 **Pratik İpucu:** Dosya yolunu tek tek yazmak yerine, Terminale komutu (örneğin sha256sum) yazıp bir boşluk bıraktıktan sonra **dosyayı sürükleyip terminal penceresinin içine bırakabilirsiniz**. Dosya yolu otomatik olarak eklenecektir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Erişilebilirlik (Availability)

Erişilebilirlik (Availability), bilgiye, sistemlere veya hizmetlere **ihtiyaç duyulduğu anda** yetkili kullanıcılar tarafından sorunsuz bir şekilde ulaşıla bilmesini ve kullanılabilmesini sağlayan temel güvenlik ilkesidir.

Gizlilik ve bütünlük sağlansa bile, sistem çalışmıyorsa o bilginin bir anlamı kalmaz.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Erişilebilirlik (Availability)

ATM'ye gittiniz. Ancak ATM ekranında "Sistem Bakımda" yazıyor veya cihaz tamamen kapanmış durumda. Burada paranız güvendedir (gizlilik tamam) ve hesabınızdaki tutar doğrudur (bütünlük tamam), ancak sisteme **erişemediğiniz** için erişilebilirlik ilkesi aksamıştır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Erişilebilirlik (Availability)

DDoS (Dağıtık Hizmet Engelleme) Saldırıları

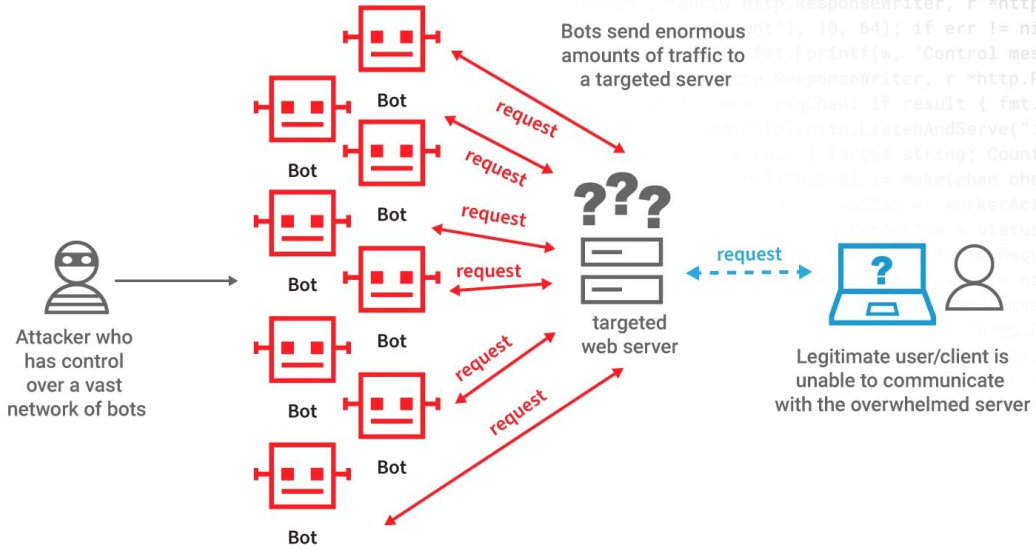
Çok popüler bir e-ticaret sitesinin indirim gününde olduğunu düşünün.

Kötü niyetli kişiler, botnet ağlarını kullanarak saniyede milyonlarca sahte istek gönderirler. Sunucu bu yoğun trafik karşısında yanıt veremez hale gelir, çöker veya kitlenir.

Gerçek müşteriler alışveriş yapmak için siteye girmek istediğinde "Sayfaya ulaşılamıyor" hatası alır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

DDOS SALDIRISI



What is volumetric attack?

Akamai

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Olmayan Kodun Maliyeti



Güvenli Olmayan Kodun Maliyeti

Güvenli olmayan kod (insecure code), yazılım geliştirme sürecinde güvenlik açıklarının bilinçli veya bilinçsiz şekilde bırakılmasıdır. Bunun maliyeti çok boyutludur:

- ❌ Doğrudan Maliyetler
- ❌ Dolaylı Maliyetler

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Doğrudan maliyetler (IBM Cost of a Data Breach raporları):

- Küresel ortalama bir veri ihlali maliyeti **4,4 – 5 milyon dolar** civarındadır (2025–2026 verileri).
- ABD'de bu rakam **11+ milyon dolara** çıkabilmektedir.
- Geliştirme aşamasında bulunan bir açığı düzeltmek, üretim ortamında düzeltmeye göre **30 kata kadar daha ucuzdur** (NIST araştırmaları).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Dolaylı maliyetler (IBM Cost of a Data Breach raporları):

- ❌ İtibar kaybı ve müşteri güveninin sarsılması
- ❌ Yasal cezalar (KVKK, GDPR, Siber Güvenlik Kanunu vb.)
- ❌ Operasyonel kesintiler ve gelir kaybı
- ❌ Teknik borç birikimi (technical debt)
- ❌ Yeniden yazma veya mimari değişiklik zorunluluğu

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Dolaylı maliyetler (IBM Cost of a Data Breach raporları):

- ❌ İtibar kaybı ve müşteri güveninin sarsılması
- ❌ Yasal cezalar (KVKK, GDPR, Siber Güvenlik Kanunu vb.)
- ❌ Operasyonel kesintiler ve gelir kaybı
- ❌ Teknik borç birikimi (technical debt)
- ❌ Yeniden yazma veya mimari değişiklik zorunluluğu

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Bilinen En Büyük Güvenlik İhlalleri Örnekleri

Olay	Yıl	Etkilenen	Temel Neden	Tahmini Maliyet / Etki
Yahoo	2013–2014	3 milyar hesap	State-sponsored saldırı + zayıf güvenlik	Verizon satışında 350 milyon $
Equifax	2017	~147–163 milyon kişi	Yamanmamış Apache Struts açığı	1,4 milyar $ üzeri (ceza + tazminat)
NotPetya	2017	Küresel	Tedarik zinciri + yamanmamış sistemler	~10 milyar $ (en pahalı siber olaylardan)
Marriott (Starwood)	2018	500 milyon	Eski sistemlerde uzun süreli yetkisiz erişim	Yüzlerce milyon $
Facebook / Meta	2019	533 milyon	Veri sızdırma / scraping	5 milyar $ FTC cezası + diğer cezalar
Change Healthcare	2024	190 milyon	Ransomware + MFA eksikliği	2,45 milyar $ üzeri
National Public Data	2024	~2,9 milyar kayıt	Yanlış yapılandırılmış veritabanı	Çok yüksek (henüz netleşmedi)
MOVEit (Progress)	2023	95+ milyon	Zero-day SQL Injection (tedarik zinciri)	Toplam ~15,8 milyar $ tahmin

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

MOVEit Zero Day Açığı

- **MOVEit sıfır gün (zero-day) açığı olayı**, Mayıs 2023 sonunda ortaya çıkan, dünya genelinde binlerce kurumu ve milyonlarca kişiyi etkileyen çok büyük bir siber güvenlik ve veri ihlali krizidir.
- **Güvenlik Açığı (CVE-2023-34362):** MOVEit Transfer adlı güvenli dosya transfer yazılımında tespit edilen kritik bir **SQL enjeksiyonu** açığıdır.
- **Etkisi:** Bu açık sayesinde saldırganlar, savunmasız sistemlere yetkisiz erişim sağlayarak veritabanlarındaki hassas bilgileri doğrudan çalmayı başardı.
- **Fail (CLOP Çetesi):** Saldırıların arkasında Rusya bağlantılı olduğu bilinen **CLOP (Clop)** isimli fidye yazılımı (ransomware) grubu yer aldı. Grup, sistemlere **LEMURLOOT** adı verilen özel bir web kabuğu (webshell) yerleştirerek veri sızdırdı.
- **Bilanço:** Küresel çapta 2.700'den fazla kuruluş (devlet kurumları, büyük bankalar, havayolları ve üniversiteler) etkilendi ve tahmini **60 milyon kişinin** kişisel verisi sızdırıldı veya ele geçirildi.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Olmayan Kodun Maliyeti

Yazılım geliştirme sürecinde "**Nasıl olsa çalışıyor, sonra düzeltiriz**" veya "Hızlıca yayına alalım" mantığı, şirketler ve geliştiriciler için yıkıcı maliyetler doğurur. Bu maliyetler yalnızca parasal değildir:

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Olmayan Kodun Maliyeti

- **Finansal Maliyetler:** Veri sızıntısı durumunda ödenen devasa yasal cezalar (örneğin KVKK, GDPR ihlal cezaları), toplu davalar (lawsuits), sistem kesintilerinin (downtime) getirdiği ticari zararlar ve acil yama/kurtarma operasyonları.
- **İtibar Maliyeti:** Şirketin müşterileri nezdindeki güveninin sarsılması. Bir kez kaybedilen güvenin geri getirilmesi, yıllar sürebilir ve milyonlarca dolarlık pazarlama bütçesi gerektirebilir.
- **Zaman ve Efor Maliyeti (Shift-Left Ekonomisi):** Yazılım mühendisliğinde altın bir kural vardır: ***Bir hata üretim ortamında (production) ne kadar geç fark edilirse, maliyeti o kadar katlanır.***
  - Tasarım aşamasında tespit edilen bir hatanın maliyeti 1 birimse, kodlama aşamasında 5 birim, test aşamasında 15 birim, üretimde (canlıda) ise **100 katına kadar** çıkabilir. Çünkü canlı sistemde hem hatayı bulmak, hem yamalamak hem de sızılan verileri/etkileri temizlemek zorundadır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

"Security by Design"
(Tasarimda Güvenlik) Felsefesi



"Security by Design" (Tasarımda Güvenlik) Felsefesi

Geleneksel yaklaşımda güvenlik, yazılım bittikten sonra "Test aşamasında bir de pentest (sızma testi) yapalım" mantığıyla ele alınır (Buna **"Bolted-on Security" / Sonradan Vidalanmış Güvenlik** denir). Ancak bu yaklaşım yetersizdir.

Security by Design ise güvenliğin yazılımın fikrini çizdiğiniz ilk kağıt üstü tasarım aşamasından itibaren mimari bir karar olarak ele alınmasıdır (**"Baked-in Security" / İçselleştirilmiş Güvenlik**).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

"Security by Design" (Tasarımda Güvenlik) Felsefesi

Security by Design, güvenliği yazılımın/ürünün "sonradan eklenen bir katmanı" değil, **en başından itibaren temel bir tasarım ilkesi** haline getirme yaklaşımıdır.

Temel Felsefe

"Güvenliği sonradan yama olarak eklemek yerine, sistemi baştan güvenli olacak şekilde tasarla."

Bu yaklaşım "shift-left" (güvenliği geliştirme sürecinin soluna, yani erken aşamalarına kaydırma) prensibiyle de örtüşür.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

"Security by Design" Temel Prensipleri Nelerdir?

1. **Threat Modeling (Tehdit Modelleme)**
   Tasarım aşamasında "Bu sistem nasıl kötüye kullanılabilir?" sorusu sorulur.
2. **Least Privilege (En Az Ayrıcalık)**
   Her bileşen yalnızca ihtiyacı olan yetkiye sahip olur.
3. **Secure by Default (Varsayılan olarak Güvenli)**
   Ürün kutudan çıktığında en güvenli ayarlarla gelir. Kullanıcı güvenliği "açmak" zorunda kalmaz.
4. **Defense in Depth (Derinlemesine Savunma)**
   Tek bir güvenlik katmanına güvenilmez; katmanlı koruma uygulanır.
5. **Assume Breach (İhlal Varsayımı)**
   Saldırganın bir şekilde içeri girebileceği kabul edilerek hasar alanı sınırlandırılır.
6. **Fail Securely**
   Hata durumunda sistem güvensiz bir duruma düşmez.
7. **Minimize Attack Surface**
   Gereksiz özellikler, servisler ve arayüzler kapatılır.

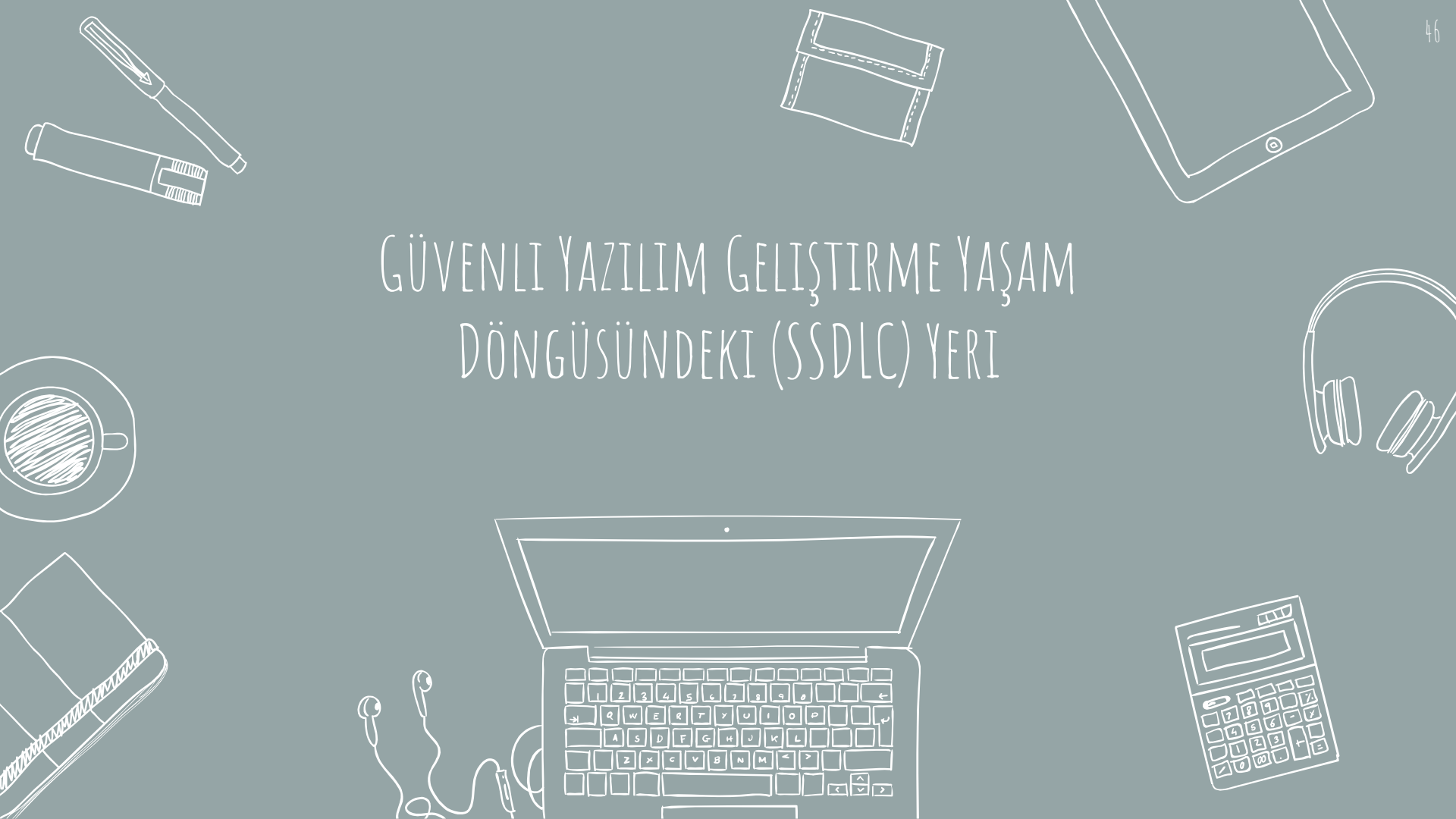
Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Neden Önemli?

- **Maliyet Avantajı:** Yazılım canlıya geçtikten sonra bulunan bir güvenlik açığının maliyeti, tasarım aşamasında fark edilip çözülen bir açığa kıyasla onlarca kat daha yüksektir.
- **Saldırı Yüzeyini Azaltma:** Gereksiz servislerin, yetkilerin ve karmaşık yapıların elenmesiyle sistemin dışarıdan gelebilecek saldırılara karşı açık noktaları minimize edilir.
- **Sürdürülebilir Güvenlik:** Güvenlik kişisel tercihlere değil, sistemsel bir süreç ve mimari standart haline gelir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Yazılım Geliştirme Yaşam Döngüsündeki (SSDLC) Yeri



Neden Önemli?

1. **Gereksinim Analizi:**
   - İşlevsel gereksinimlerin yanında **güvenlik gereksinimleri** (KVKK/GDPR uyumluluğu, veri gizliliği, erişim denetimleri) belirlenir.

2. **Mimari ve Tasarım:**
   - **Tehdit Modelleme (Threat Modeling):** Sistem mimarisi üzerindeki olası veri akışları analiz edilerek saldırı yüzeyleri ve olası tehditler (STRIDE vb. yöntemlerle) henüz kod yazılmadan tespit edilir.
   - Güvenli veri depolama ve iletişim protokolleri seçilir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Neden Önemli?

3. **Kodlama ve Geliştirme:**
   - Güvenli kodlama standartları (OWASP ilkeleri, girdi doğrulama, çıktı kodlama) uygulanır.
   - Statik Kod Analizi (SAST) araçları ile zafiyetler anlık olarak tespit edilir.
4. **Test ve Doğrulama:**
   - Dinamik Güvenlik Testleri (DAST), Bağımlılık/Kütüphane Taramaları (SCA) ve Sızma Testleri (Penetration Testing) ile tasarımın doğrulaması yapılır.
5. **Bakım ve Operasyon:**
   - Canlı sistemde izleme, günlük kaydı (logging) ve hızlı yama mekanizmaları (incident response) aktif tutulur.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com





Thanks!

Any questions?

You can find me at:
@username
user@mail.me









